# OPERATING SYSTEM PRACTICAL

**PRACTICAL-1**

Aim: Write a Program to Give solution to the Producer-Consumer Problem using Shared Memory.

**Input:**

```
import threading
import queue
import time
import random

BufferSize = 5
buffer = queue.Queue(BufferSize)

ItemToProduce = 5

def producer():
    for item in range(1, ItemToProduce + 1):
        buffer.put(item)
        print(f"Producer produced: ",item)
        time.sleep(random.uniform(0.2, 0.8))

def consumer():
    for _ in range(ItemToProduce):
        item = buffer.get()
        print(f"Consumer consumed: ",item)
        time.sleep(random.uniform(0.3, 1.0))

producer_thread = threading.Thread(target=producer)
consumer_thread = threading.Thread(target=consumer)

producer_thread.start()
consumer_thread.start()

producer_thread.join()
consumer_thread.join()

print("\nProducer-Consumer task completed.")
```

**Output:**

```
Producer produced:  1
Consumer consumed:  1
Producer produced:  2
Consumer consumed:  2
Producer produced:  3
Consumer consumed:  3
Producer produced:  4
Consumer consumed:  4
Producer produced:  5
Consumer consumed:  5

Producer-Consumer task completed.
```

**PRACTICAL-2**

Aim: Write a python program for process communication using message passing.

**Input:**

```
from multiprocessing import Process, Pipe

# Function for Process 1 (Sender)
def sender(conn):
    messages = ["Hello", "How are you?", "This is message passing", "exit"]
    for msg in messages:
        print(f"Sender: Sending -> {msg}")
        conn.send(msg)
    conn.close()

# Function for Process 2 (Receiver)
def receiver(conn):
    while True:
        msg = conn.recv()
        print(f"Receiver: Got -> {msg}")
        if msg == "exit":
            break
    conn.close()

if __name__ == "__main__":
    parent_conn, child_conn = Pipe()

    p1 = Process(target=sender, args=(parent_conn,))
    p2 = Process(target=receiver, args=(child_conn,))

    p1.start()
    p2.start()

    p1.join()
    p2.join()

    print("Message passing between processes completed.")
```

**Output:**

```
Sender: Sending -> Hello
Sender: Sending -> How are you?
Sender: Sending -> This is message passing
Sender: Sending -> exit
Receiver: Got -> Hello
Receiver: Got -> How are you?
Receiver: Got -> This is message passing
Receiver: Got -> exit
Message passing between processes completed.
```

**PRACTICAL-3**

Aim: Write a python program to work with Single thread.

**Input:**

```
import time

def task1():
    print("Task 1: Start printing numbers")
    for i in range(1, 6):
        print(f"Task 1 -> {i}")
        time.sleep(1)
    print("Task 1: Completed\n")

def task2():
    print("Task 2: Start printing letters")
    for ch in ['A', 'B', 'C', 'D', 'E']:
        print(f"Task 2 -> {ch}")
        time.sleep(1)
    print("Task 2: Completed\n")

if __name__ == "__main__":
    start = time.time()

    task1()
    task2()

    end = time.time()
    print(f"All tasks completed in {end - start:.2f} seconds.")
```

**Output:**

```
Task 1: Start printing numbers
Task 1 -> 1
Task 1 -> 2
Task 1 -> 3
Task 1 -> 4
Task 1 -> 5
Task 1: Completed

Task 2: Start printing letters
Task 2 -> A
Task 2 -> B
Task 2 -> C
Task 2 -> D
Task 2 -> E
Task 2: Completed

All tasks completed in 10.00 seconds.
```

**PRACTICAL-4**

Aim: Write a python program to work with Multi thread.

**Input:**

```
import threading
import time

def task1():
    for i in range(1, 6):
        print(f"Task 1 → {i}")
        time.sleep(1)

def task2():
    for ch in ['A', 'B', 'C', 'D', 'E']:
        print(f"Task 2 → {ch}")
        time.sleep(1)

t1 = threading.Thread(target=task1)
t2 = threading.Thread(target=task2)

t1.start()
t2.start()

t1.join()
t2.join()

print("\nAll tasks completed.")
```

**Output:**

```
Task 1 → 1
Task 2 → A
Task 1 → 2
Task 2 → B
Task 1 → 3
Task 2 → C
Task 1 → 4
Task 2 → D
Task 1 → 5
Task 2 → E

All tasks completed.
```

**PRACTICAL-5**

Aim: Write a python program to give a solution to the Bounded Buffer problem.

**Input:**

```
import threading
import random
import time

x = 0
lock = threading.Lock()

def Reader():
    global x
    with lock:
        print("Reader is Reading!")
        print('Shared Data: ', x)
    print()

def Writer():
    global x
    with lock:
        print('Writer is Writing!')
        x += random.randint(1, 10)
        print('Updated Shared Data to: ', x)
    print()

if __name__ == '__main__':
    threads = []

    for i in range(10):
        if random.randint(0, 100) > 50:
            t = threading.Thread(target=Reader)
        else:
            t = threading.Thread(target=Writer)
        t.start()
        threads.append(t)
        time.sleep(0.1)
    for t in threads:
        t.join()
```

**Output:**

```
Writer is Writing!
Updated Shared Data to:  7

Reader is Reading!
Shared Data:  7

Writer is Writing!
Updated Shared Data to:  13
```

**PRACTICAL-6**

Aim: Write a python program to give a solution to the Reader-Writers Problem.

**Input:**

```
import threading
import random
import time

x = 0
lock = threading.Lock()

def reader():
    global x
    print("Reader is reading...")
    lock.acquire()
    print("Shared Data:", x)
    lock.release()
    print()

def writer():
    global x
    print("Writer is writing...")
    lock.acquire()
    x += 1
    print("Writer updated data to", x)
    lock.release()
    print()

if __name__ == "__main__":
    threads = []

    for i in range(10):
        if random.randint(0, 100) > 50:
            t = threading.Thread(target=reader)
        else:
            t = threading.Thread(target=writer)
        t.start()
        threads.append(t)
        time.sleep(0.2)

    for t in threads:
        t.join()

    print("Reader–Writer simulation completed.")
```

**PRACTICAL-7**

Aim: Write a python program to give a semaphore solution to the dining philosopher program.

**Input:**

```
import threading
import time
import random

num_philosophers = 5
num_forks = num_philosophers

forks = [threading.Semaphore(1) for i in range(num_forks)]
mutex = threading.Semaphore(1)

def philosopher(index):
    while True:
        print(f"Philosopher {index} is thinking...")
        time.sleep(random.randint(1, 5))

        mutex.acquire()

        left_fork_index = index
        right_fork_index = (index + 1) % num_forks

        forks[left_fork_index].acquire()
        forks[right_fork_index].acquire()

        mutex.release()

        print(f"Philosopher {index} is eating...")
        time.sleep(random.randint(1, 5))

        forks[left_fork_index].release()
        forks[right_fork_index].release()

philosopher_threads = []
for i in range(num_philosophers):
    philosopher_threads.append(threading.Thread(target=philosopher, args=(i,)))

for thread in philosopher_threads:
    thread.start()

for thread in philosopher_threads:
    thread.join()
```

**PRACTICAL-8**

Aim: Write a Program to Implements CPU Scheduling Algorithm.

**Input:**

```
def findWaitingTime(processes, n, bt, wt):
    wt[0] = 0
    for i in range(1, n):
        wt[i] = bt[i-1] + wt[i-1]

def findTurnAroundTime(processes, n, bt, wt, tat):
    for i in range(n):
        tat[i] = bt[i] + wt[i]

def findAvgTime(processes, n, bt):
    wt = [0] * n
    tat = [0] * n
    total_wt = 0
    total_tat = 0

    findWaitingTime(processes, n, bt, wt)
    findTurnAroundTime(processes, n, bt, wt, tat)

    print("Process\tBurst Time\tWaiting Time\tTurn Around Time")
    for i in range(n):
        total_wt += wt[i]
        total_tat += tat[i]
        print(f"  {processes[i]}\t\t\t{bt[i]}\t\t\t\t{wt[i]}\t\t\t\t\t{tat[i]}")

    print(f"\nAverage waiting time = {total_wt / n:.2f}")
    print(f"Average turnaround time = {total_tat / n:.2f}")

if __name__ == '__main__':
    processes = [1, 2, 3]
    n = len(processes)
    burst_time = [10, 5, 8]

    findAvgTime(processes, n, burst_time)
```

**PRACTICAL-9**

Aim: Write a Program to Implements Banker's Algorithm.

**Input:**

```
P = 5  # Number of processes
R = 3  # Number of resources

def calculate_need(maxm, allot):
    need = [[maxm[i][j] - allot[i][j] for j in range(R)] for i in range(P)]
    return need

def is_safe(processes, avail, maxm, allot):
    need = calculate_need(maxm, allot)
    finish = [0] * P
    safe_seq = []
    work = avail.copy()

    count = 0
    while count < P:
        found = False
        for p in range(P):
            if not finish[p] and all(need[p][j] <= work[j] for j in range(R)):

                for j in range(R):
                    work[j] += allot[p][j]
                safe_seq.append(p)
                finish[p] = 1
                count += 1
                found = True
        if not found:
            print("System is NOT in a safe state.")
            return False

    print("System is in a safe state.")
    print("Safe sequence:", ' -> '.join(f"P{p}" for p in safe_seq))
    return True

if __name__ == "__main__":
    processes = [0, 1, 2, 3, 4]

    avail = [3, 3, 2]
    maxm = [[7, 5, 3],[3, 2, 2],[9, 0, 2],[2, 2, 2],[4, 3, 3]]
    allot = [[0, 1, 0],[2, 0, 0],[3, 0, 2],[2, 1, 1],[0, 0, 2]]

    is_safe(processes, avail, maxm, allot)
```

**PRACTICAL-10**

Aim: Write a python program to implement LRU page replacement algorithm.

**Input:**

```
size=3

refrence_string=[1,2,1,0,3,4,2,4]
pages=[]
fault=0
hits=0

for ref_page in refrence_string:
    if ref_page in pages:
        pages.remove(ref_page)
        pages.append(ref_page)
        hits+=1
    else:
        fault+=1
        if (len(pages)<size):
            pages.append(ref_page)
        else:
            pages.remove(pages[0])
            pages.append(ref_page)

print("Total number of Page Hits: ",hits)
print("Total number of Page Faults: ",fault)
```

**Output:**

```
Total number of Page Hits:  2
Total number of Page Faults:  6
```